

Informe Tecnico del Proyecto Overflod

Programacion Web - UAIN1349P

Aplicacion web para emprendimiento local con HTML, CSS, JavaScript, Bootstrap, Node.js, Express.js y base de datos compatible con SQL Server.

1.1 Introduccion y descripcion del problema

Overflod es un emprendimiento dedicado a la venta de materiales de construccion como cemento, fierro, ladrillos, agregados, PVC, pintura, ferreteria y sanitarios. Su necesidad principal es digitalizar procesos que normalmente se atienden por llamada, WhatsApp o de manera presencial: consulta de productos, cotizaciones, pedidos, control de inventario y seguimiento de clientes.

El problema identificado es que un cliente no siempre puede conocer disponibilidad, precios y productos antes de comunicarse con la tienda. A la vez, el administrador necesita una herramienta simple para registrar ventas, responder consultas, controlar stock y registrar compras a proveedores.

Perfiles de usuario:

- Cliente: busca productos, revisa precio, se registra, arma carrito y confirma pedidos.
- Visitante: revisa informacion general y envia consultas desde contacto.
- Administrador: gestiona inventario, pedidos, ventas, clientes, mensajes y compras a proveedores.
- Proveedor: abastece productos, costos y cantidades que se registran en el panel administrador.

1.2 Justificacion tecnica y etica

Objetivo general: implementar una aplicacion web funcional, responsiva y accesible para sistematizar procesos comerciales de Overflod.

Objetivos especificos:

- Mostrar un catalogo filtrable de productos.
- Permitir registro, inicio de sesion y pedidos.
- Registrar mensajes de contacto.
- Gestionar inventario, proveedores, compras, ventas, clientes y pedidos pendientes.
- Conectar el sistema a un backend Node.js + Express.js compatible con SQL Server.

Mapa de empatia del cliente:

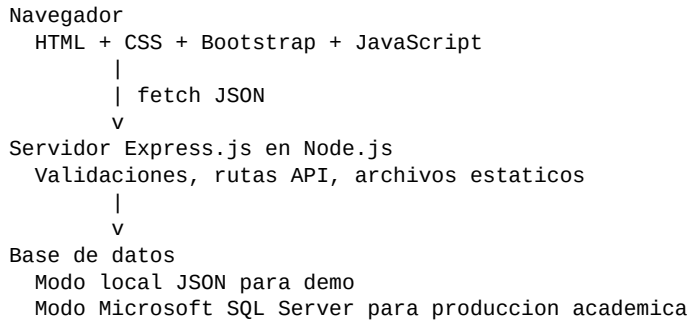
- Piensa: necesita precios claros, stock disponible y rapidez.
- Siente: desconfianza si no hay informacion ordenada o respuesta rapida.
- Ve: negocios competidores que ya publican productos en internet.
- Oye: recomendaciones de maestros de obra, familiares y proveedores.
- Dice y hace: compara precios, consulta por WhatsApp y busca disponibilidad.
- Dolores: perdida de tiempo, falta de stock, precios poco claros.
- Ganancias: compra mas rapida, historial de pedidos y contacto directo.

Consideraciones eticas:

- Se solicitan solo datos necesarios: nombre, correo, telefono y contrasena.
- No se muestran contrasenas en el panel administrador.
- Los mensajes se gestionan internamente.
- El diseno prioriza lectura clara y uso simple para usuarios con distintos niveles digitales.

1.3 Arquitectura y tecnologias empleadas

Arquitectura cliente-servidor:



Tecnologias:

- HTML5 semantico.
- CSS3 modular.
- Bootstrap 5.3.
- JavaScript para DOM, eventos, validaciones y consumo API.
- Node.js.
- Express.js.
- SQL Server compatible mediante `mssql`.
- JSON local para demostracion sin instalar SQL Server.

Flujo de navegacion:

```

Inicio -> Catalogo -> Detalle -> Carrito -> Login/Registro -> Pedido
Inicio -> Contacto -> Mensaje -> Admin -> Mensajes
Admin -> Inventario -> Compras proveedor -> Stock actualizado
Admin -> Pedidos -> Estado -> Ventas/Clientes

```

Wireframes textuales:

- Inicio: navbar, hero, accesos a catalogo/compras/admin.
- Catalogo: filtros superiores, grilla de productos, modal detalle, carrito lateral.
- Contacto: datos de atencion y formulario.
- Perfil: acciones de cuenta e historial.
- Admin: resumen, metricas, tabs y tablas de gestion.

1.4 Descripcion de funcionalidades implementadas

Front-end:

- Paginas responsivas.
- Catalogo con imagenes referenciales.
- Filtros por categoria, marca, material y precio.
- Carrito con cantidades.
- Modales de login, registro y detalle.
- FAQ filtrable.
- Formulario de contacto validado.
- Panel administrador con inventario, compras, ventas, clientes, pedidos y mensajes.

Back-end:

- `GET /api/bootstrap`: carga datos iniciales.
- `GET /api/products`: lista productos.
- `POST /api/products`: crea producto.
- `PATCH /api/products/:id`: actualiza stock, costo, precio u otros campos.
- `POST /api/register`: registra cliente.
- `POST /api/login`: autentica usuario.
- `POST /api/orders`: crea pedido y descuenta inventario.
- `PATCH /api/orders/:id/status`: cambia estado de pedido.
- `POST /api/queries`: registra mensaje de contacto.
- `PATCH /api/queries/:id/respond`: marca consulta como respondida.
- `POST /api/purchases`: registra compra a proveedor y suma stock.

Validaciones:

- Correos con formato valido.
- Telefonos con longitud y caracteres permitidos.
- Contraseñas con minimo de caracteres.
- Stock entero no negativo.
- Precio mayor que cero.
- Precio de venta no menor al costo de compra.
- Cantidades de compra y pedido mayores que cero.
- Estados permitidos para pedidos y mensajes.

1.5 Buenas practicas aplicadas

Accesibilidad:

- Etiquetas `aria-label` en botones iconicos o de cierre.
- `aria-live` para contenido dinamico.
- `aria-labelledby` en modales y carrito lateral.
- Enlace para saltar al contenido principal.
- Labels asociados a inputs.
- Texto alternativo en imagenes.

- Contraste y foco visible en elementos interactivos.

Privacidad:

- El panel de usuarios no expone contraseñas.
- Los datos de contacto se limitan a lo necesario.

Diseño responsivo:

- Navbar colapsable.
- Grillas Bootstrap para catálogo, métricas y formularios.
- Media queries para móviles.

Validación con usuarios:

- Se recomienda validar con un cliente real y un administrador del emprendimiento.
- Pruebas sugeridas: encontrar un producto, registrarse, confirmar pedido y enviar consulta.

1.6 Conclusiones y aprendizajes

El proyecto permite que Overflod pase de una atención dispersa a una plataforma organizada para ventas y gestión interna. La solución integra front-end responsivo, interactividad con JavaScript y back-end con Express.js, cumpliendo el enfoque cliente-servidor del curso.

Aprendizajes:

- La validación debe realizarse en cliente y servidor.
- Un panel administrador aporta valor cuando conecta ventas, inventario y proveedores.
- La accesibilidad mejora la experiencia de todos los usuarios, no solo de personas con discapacidad.
- Separar modo local y SQL Server permite desarrollar rápido y desplegar en entornos académicos o reales.

Mejoras futuras:

- Hash de contraseñas.
- Roles más granulares.
- Reportes descargables.
- Integración con pasarela de pagos.
- Envío automático de correos de confirmación.